Contrato

En la construcción de software, un **contrato** (conocido formalmente como **Diseño por Contrato** o *Design by Contract*) es un modelo de diseño que establece un acuerdo formal, estricto e irrompible entre un componente de software (el proveedor) y el código que lo llama (el cliente).

Este concepto fue inventado por **Bertrand Meyer** al crear el lenguaje de programación Eiffel. La analogía con el mundo real es perfecta: un contrato legal dice *"Si tú cumples con la condición X, yo me comprometo a entregarte el resultado Y"*. En el software funciona exactamente igual para garantizar que el código sea seguro y libre de errores.

Un contrato de software se compone de **tres elementos fundamentales**:

Los 3 pilares de un contrato de software

1. Precondiciones (Lo que el cliente debe cumplir)

Son los requisitos obligatorios que el código que llama al método debe garantizar **antes** de ejecutarlo. Si la precondición no se cumple, el método se niega a trabajar y suele lanzar un error inmediatamente.

- **Ejemplo:** Para una función `retirar_dinero(monto)`, la precondición es que el `monto` sea mayor a cero y menor o igual al saldo disponible.

2. Postcondiciones (Lo que el método promete entregar)

Es la garantía de lo que el método habrá realizado **después** de su ejecución, siempre y cuando las precondiciones se hayan cumplido.

- **Ejemplo:** Para la misma función `retirar_dinero(monto)`, la postcondición es que el saldo final sea exactamente el `saldo_inicial - monto` y que se retorne el dinero.

3. Invariantes (Lo que nunca debe cambiar)

Son condiciones globales que **deben permanecer verdaderas en todo momento**, tanto antes como después de ejecutar cualquier método del componente o clase.

- **Ejemplo:** En una cuenta bancaria, el invariante podría ser que el `saldo` nunca puede ser un número negativo, o que el `número_de_cuenta` no puede cambiar.

Ejemplo

Mira cómo se traduce esto en un lenguaje común como Python utilizando aserciones (`assert`) para validar el contrato:

```
class CuentaBancaria:
    def __init__(self, saldo_inicial: float):
        self.saldo = saldo_inicial

    def verificar_invariante(self):
        # INVARIANTE: El saldo jamás puede ser negativo en este tipo de cuenta
        assert self.saldo >= 0, "Violación de Invariante: Saldo negativo"

    def retirar_dinero(self, monto: float) -> float:
        # 1. PRECONDICIONES (Responsabilidad del cliente que llama a la función)
        assert monto > 0, "Precondición fallida: El monto debe ser positivo"
```

```
        assert monto <= self.saldo, "Precondición fallida: Fondos insuficientes"

        saldo_anterior = self.saldo

        # Ejecución del trabajo
        self.saldo -= monto

        # 2. POSTCONDICIONES (Responsabilidad de esta función)
        assert self.saldo == saldo_anterior - monto, "Postcondición fallida:
Error en el cálculo del saldo"

        # 3. VERIFICAR INVARIANTE (Asegurar la salud del objeto)
        self.verificar_invariante()

        return monto
```

¿Por qué se usan los contratos en el software?

- **Eliminan la redundancia:** Si la función exige mediante una precondición que el parámetro no sea nulo (`null`), ya no tienes que llenar el interior de la función con infinitos `if (objeto != null)` para protegerte. El cliente está obligado a mandarlo bien.
- **Depuración ultra rápida:** Si el software falla, el contrato te dice exactamente quién rompió el trato. Si falla la precondición, el error es de quien *llamó* a la función. Si falla la postcondición, el error es de la función *interna*.
- **Documentación viva:** El contrato actúa como la mejor documentación posible para otros desarrolladores, ya que define las reglas exactas de cómo usar el código.

Tipos de cohesión (De mejor a peor)

1. Cohesión Funcional (La ideal)

La **cohesión funcional** es el nivel más alto y deseable de cohesión en la ingeniería de software. Ocurre cuando todos los elementos dentro de un módulo, componente o función están agrupados porque cooperan para realizar una única tarea bien definida.

En este nivel, el componente no hace "varias cosas relacionadas", sino que hace **una sola acción atómica**. Si intentaras dividir esa función en partes más pequeñas, destruirías su propósito principal.

Características clave

- **Propósito único:** El nombre del método suele tener un verbo claro y un objeto directo (ej. `calcular_interes()`, `validar_email()`).
- **Fácil de probar:** Al tener una sola responsabilidad, las pruebas unitarias son muy sencillas de escribir.
- **Alta reutilización:** Como no depende de contextos ocultos ni hace tareas secundarias, se puede usar en cualquier parte del sistema.

Ejemplos

1. Cálculo matemático puro

Una función que recibe datos, aplica una fórmula y regresa el resultado. No guarda en base de datos, no formatea texto, solo calcula.

```
def calcular_area_circulo(radio: float) -> float:
    if radio < 0:
        raise ValueError("El radio no puede ser negativo")
    return 3.14159 * (radio ** 2)
```

- **Por qué es funcional:** Todo el código dentro de la función existe únicamente para resolver el área de un círculo.

2. Validación de reglas de negocio

Un componente que evalúa si una cadena de texto cumple con una estructura específica.

```
//typescript
function esEmailValido(email: string): boolean {
    const expresionRegular = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return expresionRegular.test(email);
}
```

- **Por qué es funcional:** Su única misión en la vida es responder "sí" o "no" a si el texto es un correo válido. No envía correos ni altera la interfaz de usuario.

3. Transformación de datos pura

Un método que toma un objeto y lo transforma en otro formato.

```
//javascript
function formatearAFechaLocal(fechaIso) {
    const fecha = new Date(fechaIso);
    return fecha.toLocaleDateString('es-ES');
}
```

- **Por qué es funcional:** Solo se encarga de la conversión de formato.

El contraejemplo: ¿Cómo se daña la cohesión funcional?

A veces los desarrolladores añaden "pequeñas tareas extras" que arruinan la cohesión funcional. Mira este caso:

```
# COHESIÓN BAJA (Mezcla de responsabilidades)
def calcular_total_pedido(productos):
    total = 0
    for p in productos:
        total += p.precio

    # MAL: Esto rompe la cohesión funcional. La función ya no solo calcula.
    guardar_log_en_archivo(f"Total calculado: {total}")
    actualizar_pantalla_usuario(total)

    return total
```

Para recuperar la **cohesión funcional**, las acciones de guardar el log y actualizar la pantalla deberían extraerse a sus propias funciones independientes, dejando a `calcular_total_pedido` haciendo exclusivamente su cálculo.

2. Cohesión Secuencial

La **cohesión secuencial** ocurre cuando un módulo o función realiza varias tareas distintas, pero están encadenadas de forma obligatoria: la salida de una tarea es la entrada de la siguiente.

Imagina una línea de ensamblaje de automóviles: no puedes pintar el coche antes de construir el chasis. En el software, este tipo de cohesión es común cuando implementas **flujos de trabajo, tuberías de datos (pipelines) o procesos de transformación paso a paso**.

Aunque no es tan perfecta como la cohesión funcional (porque el módulo hace *más de una cosa*), se considera una **cohesión buena y aceptable** porque las tareas están íntimamente ligadas por los datos.

Características clave

- **Dependencia de datos:** El paso B no puede iniciar si el paso A no ha terminado y entregado sus resultados.
- **Orden estricto:** El orden de las operaciones no se puede alterar bajo ninguna circunstancia.
- **Flujo lineal:** El código se lee de arriba hacia abajo como una receta de cocina.

Ejemplos

1. Procesamiento de archivos (Pipeline de Datos)

Un caso clásico es leer un archivo crudo, limpiar su contenido y luego estructurarlo.

```
def procesar_reporte_mensual(ruta_archivo):
    # Paso 1: Obtener los datos crudos
    texto_crudo = leer_archivo_csv(ruta_archivo)

    # Paso 2: Limpiar los datos (depende del Paso 1)
    datos_limpios = eliminar_filas_duplicadas(texto_crudo)

    # Paso 3: Transformar el resultado (depende del Paso 2)
    reporte_final = generar_metricas_financieras(datos_limpios)

    return reporte_final
```

- **Por qué es secuencial:** `eliminar_filas_duplicadas` no puede trabajar sin el `texto_crudo`, y `generar_metricas_financieras` necesita obligatoriamente los `datos_limpios`.

2. Proceso de Checkout / Pago

En el comercio electrónico, el flujo de procesamiento de una orden sigue una secuencia estricta.

```
//typescript
function procesarOrdenDeCompra(carritoActual: Carrito) {
    // Paso 1: Calcular totales e impuestos
    const ordenConTotales = calcularImpuestosYEnvio(carritoActual);

    // Paso 2: Cobrar (necesita el monto exacto calculado en el paso anterior)
    const reciboPago = ejecutarCargoTarjeta(ordenConTotales);

    // Paso 3: Generar factura (necesita la confirmación del pago)
    const factura = crearFacturaLegal(reciboPago);

    return factura;
}
```

- **Por qué es secuencial:** No puedes cobrar a la tarjeta si no has calculado los impuestos, y no puedes emitir una factura legal sin el recibo de pago.

Cohesión Funcional vs. Cohesión Secuencial (La diferencia)

Criterio	Cohesión Funcional	Cohesión Secuencial
¿Cuántas tareas hace?	**Una sola** tarea atómica.	**Múltiples** tareas consecutivas.
Relación interna	Todo el código se enfoca en un único fin.	El código es una cadena; cada eslabón alimenta al siguiente.
Ejemplo rápido	`calcular_iva(monto)`	`calcular_iva` → `aplicar_descuento` → `generar_ticket`

3. Cohesión Comunicacional (o de datos)

También llamada **cohesión de datos,** ocurre cuando los elementos de un módulo o clase realizan actividades diferentes, pero están agrupados porque operan exactamente sobre el mismo conjunto de datos de entrada o salida.

A diferencia de la cohesión secuencial, aquí las tareas **no dependen entre sí**. El paso B no necesita el resultado del paso A; ambos pasos simplemente leen o modifican la misma información.

Se considera un nivel de cohesión **moderado-alto (bueno)**. Es sumamente común en la programación orientada a objetos, ya que las clases suelen agrupar métodos que modifican los mismos atributos internos.

Características clave

- **Datos compartidos:** Existe una estructura de datos central (un objeto, un registro, una fila de base de datos) que une a todas las funciones.
- **Orden independiente:** A diferencia de la cohesión secuencial, el orden en que ejecutes las funciones por lo general no altera el resultado de las demás.
- **Múltiples acciones:** El módulo hace más de una cosa, pero todas enfocadas en el mismo "sujeto".

Ejemplos

1. Un módulo de gestión de usuarios (Caso clásico)

Imagina un componente que recibe los datos de un usuario y realiza tres acciones independientes con ellos.

```
def registrar_nuevo_usuario(datos_usuario):
    # Acción A: Guardar en la base de datos
    guardar_en_base_datos(datos_usuario)

    # Acción B: Enviar un correo (No depende de que la BD responda datos)
    enviar_correo_bienvenida(datos_usuario)

    # Acción C: Crear un perfil de analíticas
    inicializar_metricas_usuario(datos_usuario)
```

- **Por qué es comunicacional:** Las tres acciones (`guardar`, `enviar_correo`, `inicializar_metricas`) son actividades totalmente distintas. Sin embargo, están juntas en este módulo porque todas consumen el mismo parámetro: `datos_usuario`.

2. Una clase de repositorio o DAO (Data Access Object)

En la programación orientada a objetos, las clases que manejan una entidad específica de la base de datos muestran este tipo de cohesión.

```
//typescript
class ProductoRepository {
    // Todos los métodos operan exclusivamente sobre la tabla/entidad "Producto"

    crearProducto(producto: Producto) { ... }

    actualizarPrecio(producto: Producto, nuevoPrecio: number) { ... }

    eliminarProducto(producto: Producto) { ... }
}
```

- **Por qué es comunicacional:** `crear`, `actualizar` y `eliminar` son operaciones conceptualmente diferentes, pero están agrupadas porque su único foco de atención son los datos del objeto `Producto`.

Comparativa: Secuencial vs. Comunicacional

Para que no quede ninguna duda, mira cómo cambia el flujo de datos:

- **Cohesión Secuencial (Flujo en cadena):**
  `Datos A` → [Función 1] → `Datos B` → [Función 2] → `Datos C`
- **Cohesión Comunicacional (Flujo en estrella):**

```
                 /--> [Acción 1: Guardar]
[Datos Usuario] ---> [Acción 2: Enviar Correo]
                 \--> [Acción 3: Generar PDF]
```

Ventajas y Desventajas

- **Lo bueno:** Mantiene el código organizado de manera lógica en torno a las entidades del negocio (ej. todo lo relacionado a "Facturas" está en un mismo sitio).
- **Lo malo:** Si la estructura de los datos compartidos cambia (por ejemplo, si renombras un campo en `datos_usuario`), todas las funciones del módulo podrían romperse al mismo tiempo.

4. Cohesión Lógica

La **cohesión lógica** ocurre cuando un módulo o función agrupa tareas que son lógicamente similares, pero funcionalmente diferentes. Para decidir cuál de todas las tareas ejecutar, el módulo suele recibir un parámetro de control (como un flag, un string o un número) y utiliza un bloque condicional (`if/else` o `switch/case`) para bifurcar el comportamiento.

A diferencia de los niveles anteriores, la cohesión lógica se considera **débil y desaconsejable (mala)**. Aunque a primera vista parece "organizado" juntar cosas parecidas, introduce serios problemas de mantenimiento y viola principios de diseño de software modernos como el **Principio de Responsabilidad Única (SRP)**.

Características clave

- **La trampa del Switch/If:** El código dentro del módulo está fragmentado por condicionales que deciden qué camino tomar.
- **Interfaz engañosa:** El usuario de la función tiene que pasarle un "mando a distancia" (parámetro de control) para decirle qué hacer.
- **Tareas acopladas:** Si quieres modificar o añadir una tarea, tienes que tocar el mismo archivo y la misma función donde están las demás tareas, aumentando el riesgo de romper código que ya funcionaba.

Ejemplos

1. La función "Multiherramienta" (Caso clásico)

Imagina una función diseñada para manejar operaciones de entrada/salida de archivos, agrupadas solo porque "todas abren archivos".

```
# COHESIÓN LÓGICA (Diseño deficiente)
def gestionar_archivo(ruta, contenido, accion):
    if accion == "LEER":
        with open(ruta, "r") as f:
            return f.read()

    elif accion == "ESCRIBIR":
        with open(ruta, "w") as f:
            f.write(contenido)

    elif accion == "ELIMINAR":
        import os
        os.remove(ruta)
```

- **Por qué es lógica:** Leer, escribir y borrar son operaciones totalmente distintas. Están juntas solo bajo el concepto lógico de "gestionar un archivo". Si necesitas cambiar cómo se borran los archivos, estás obligado a editar la función que también lee y escribe.

2. El formateador universal

Un método que se encarga de dar formato a un reporte según el tipo de cliente.

```
//typescript
// COHESIÓN LÓGICA (Diseño deficiente)
function exportarReporte(datos: any, formato: string) {
    if (formato === "PDF") {
        // 50 líneas de código para renderizar un PDF...
    } else if (formato === "EXCEL") {
        // 50 líneas de código para estructurar celdas de Excel...
    } else if (formato === "JSON") {
        // 5 líneas de código para parsear texto...
    }
}
```

- **Por qué es lógica:** Generar un PDF no tiene nada que ver a nivel técnico con generar un Excel. Comparten el concepto lógico de "exportar", pero sus implementaciones son mundos aparte. El archivo crecerá infinitamente cada vez que se añada un nuevo formato.

¿Por qué es peligrosa la Cohesión Lógica?

1. **Código enredado (Efectos secundarios):** Es común que las tareas compartan variables locales dentro de la función por error, lo que provoca que cambiar la lógica de la tarea "A" termine alterando el comportamiento de la tarea "B".
2. **Pruebas difíciles:** Para probar a fondo esta función, debes enviarle múltiples combinaciones de parámetros de control.
3. **Alto acoplamiento:** Los módulos que llaman a esta función necesitan conocer internamente qué "strings" o "flags" acepta (ej. saber que se escribe `"PDF"` y no `"pdf"`).

¿Cómo se soluciona? (Refactorización)

La solución para romper la cohesión lógica es aplicar **Cohesión Funcional**: dividir la gran función en funciones pequeñas e independientes que hagan una sola cosa.

Código corregido:

```
def leer_archivo(ruta):
    with open(ruta, "r") as f: return f.read()

def escribir_archivo(ruta, contenido):
    with open(ruta, "w") as f: f.write(contenido)
```

```
def eliminar_archivo(ruta):
    import os
    os.remove(ruta)
```

Ahora cada función tiene **cohesión funcional**, es independiente, fácil de testear y si añades una nueva operación (como `comprimir_archivo`), creas una función nueva sin tocar las anteriores.

5. Cohesión coincidente

La **cohesión coincidente** es el nivel más bajo, débil y **dañino** de cohesión en la ingeniería de software. Ocurre cuando los elementos de un módulo o clase se agrupan por pura casualidad, conveniencia o flojera, sin que exista ninguna relación lógica, funcional ni de datos entre ellos.

En este escenario, el módulo se convierte en una especie de "cajón de sastre" o "bolsa de gatos". Los componentes están ahí simplemente porque el desarrollador no sabía dónde más ponerlos o porque quería agrupar código para "reutilizarlo" sin un criterio arquitectónico.

Se considera una **pésima práctica de diseño** que destruye por completo la mantenibilidad del software.

Características clave

- **Falta total de relación:** Si intentas explicar qué hace el módulo en una sola frase, terminarás usando muchas conjunciones ("y además", "también hace").
- **Nombres genéricos:** Los archivos o clases suelen llamarse `Utilidades`, `Miscelaneo`, `Global`, `Helpers` o `Common`.
- **Código Frankenstein:** Las funciones dentro del módulo pertenecen a dominios de negocio completamente diferentes (ej. finanzas, interfaz de usuario y matemáticas juntas).

Ejemplos

1. La clase "Utilidades" descontrolada (El caso más común)

Es muy frecuente encontrar clases creadas con la buena intención de "ahorrar líneas de código", pero que terminan mezclando peras con manzanas.

```
# COHESIÓN COINCIDENTE (Diseño desastroso)
class UtilidadesGenerales:

    def validar_formato_email(self, email: string) -> bool:
        # Tarea de validación de texto / usuarios
        ...

    def calcular_interes_compuesto(self, capital: float, tasa: float) -> float:
        # Tarea puramente financiera / matemática
        ...

    def reproducir_sonido_error(self):
        # Tarea multimedia / interfaz de usuario
        ...

    def convertir_segundos_a_horas(self, segundos: int) -> str:
        # Tarea de conversión de tiempo
        ...
```

- **Por qué es coincidente:** Validar un email no tiene absolutamente nada que ver con calcular intereses financieros ni con reproducir un sonido. No comparten datos, no van en secuencia, ni pertenecen al mismo concepto lógico. Estar juntos es una mera coincidencia geográfica en el proyecto.

2. El módulo "Varios.js"

Un archivo JavaScript donde se arrojan funciones sueltas a medida que avanza el proyecto.

```
//javascript
// varios.js (COHESIÓN COINCIDENTE)
export function limpiarPantalla() {
    document.getElementById("app").innerHTML = "";
}

export function encriptarPassword(password) {
    return crypto.createHash('sha256').update(password).digest('hex');
}

export function obtenerClimaActual(ciudad) {
    return fetch(`https://clima.com{ciudad}`).then(res => res.json());
}
```

- **Por qué es coincidente:** Limpiar el HTML del navegador, encriptar una contraseña de seguridad y consultar una API externa del clima son tres mundos totalmente aislados.

¿Por qué es tan destructiva la Cohesión Coincidente?

1. **Cero reusabilidad real:** Si otro desarrollador en otra parte del sistema necesita la función de `calcular_interes_compuesto`, se verá obligado a importar la clase `UtilidadesGenerales`. Esto arrastrará dependencias innecesarias de interfaces de usuario o librerías multimedia.
2. **Imposible de organizar:** A medida que el equipo crece, nadie sabe dónde poner las funciones nuevas. El archivo crece infinitamente hasta volverse ilegible.
3. **Alto acoplamiento oculto:** Si dos funciones en este módulo empiezan a depender sutilmente entre sí por flojera de diseño, romper una romperá la otra sin una razón lógica aparente.

¿Cómo se soluciona? (Refactorización)

La solución es **diseccionar el módulo** y enviar cada función a su propio contexto aplicando **Cohesión Funcional**. Es preferible tener muchos archivos pequeños bien enfocados que un mega-archivo que lo hace todo.

- `validar_formato_email` se mueve a un módulo de `Validaciones` o `Usuario`.
- `calcular_interes_compuesto` se mueve a un módulo o servicio `Financiero`.
- `reproducir_sonido_error` se mueve a la capa de `UI` o `Notificaciones`.
- `convertir_segundos_a_horas` se mueve a un formateador de `Tiempo`.